

Validation and risk

Mike K Smith (Pfizer)

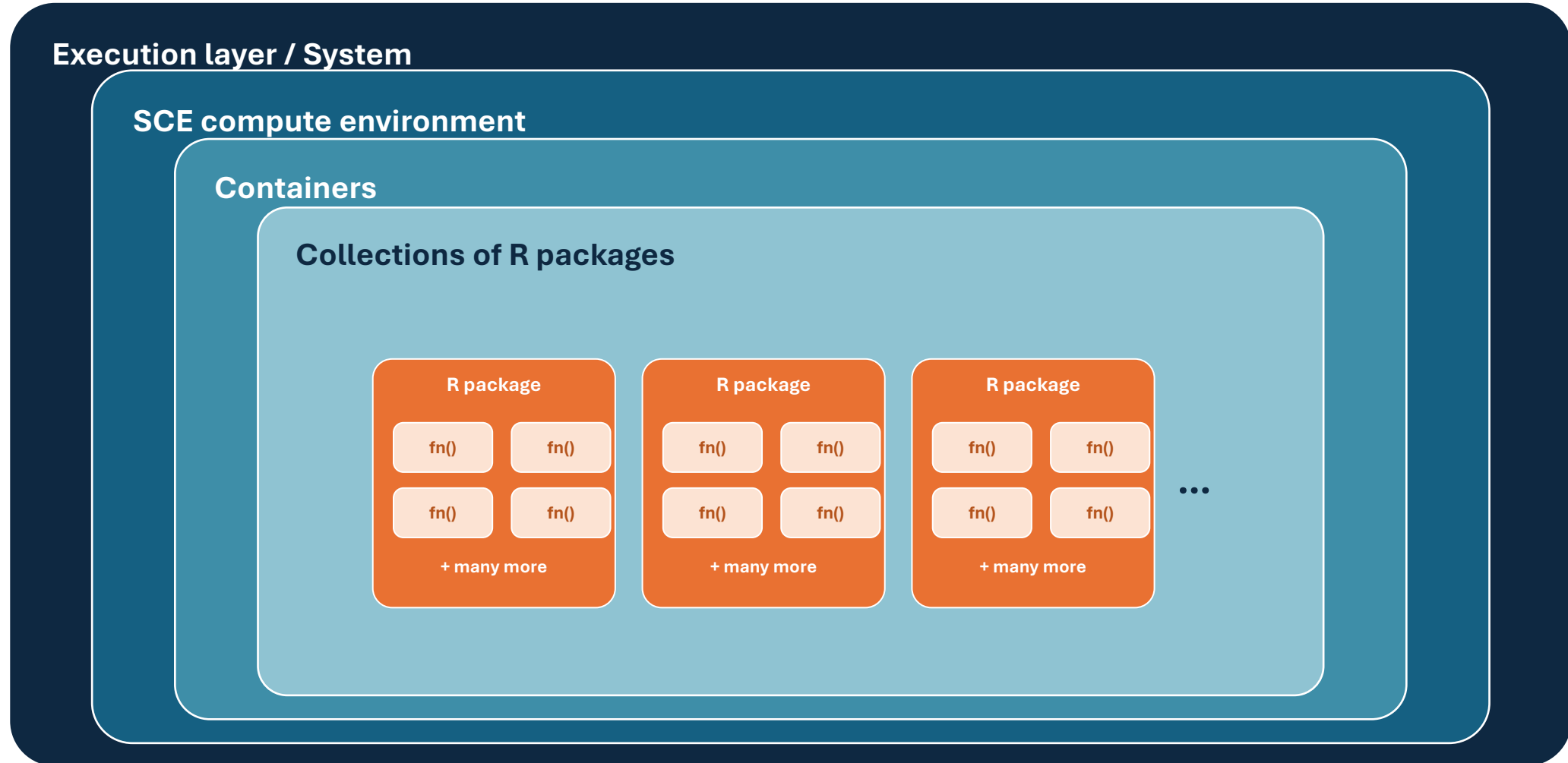
Reminder: What do you mean by “Validation”?

- “Establishing **documented evidence** which provides a high degree of assurance (**accuracy**) that a specific process consistently (**reproducibility**) produces a product meeting its predetermined specifications (**traceability**) and quality attributes.” – [*FDA Glossary of Computer System Terminology*](#)
- “The computer software used for data management and statistical analysis should be reliable, and **documentation of appropriate software testing procedures** should be available.” – [*ICH E9*](#)

Reminder: What do you mean by “Validation”?

- “The sponsor may rely on **qualification documentation** provided by the vendor, if the qualification activities performed by the vendor have been assessed as adequate. However, the sponsor may also have to **perform additional qualification (and validation) activities based on a documented risk assessment.**” – EMA [Notice to sponsors on validation and qualification of computerised systems used in clinical trials and the associated Q&A.](#)

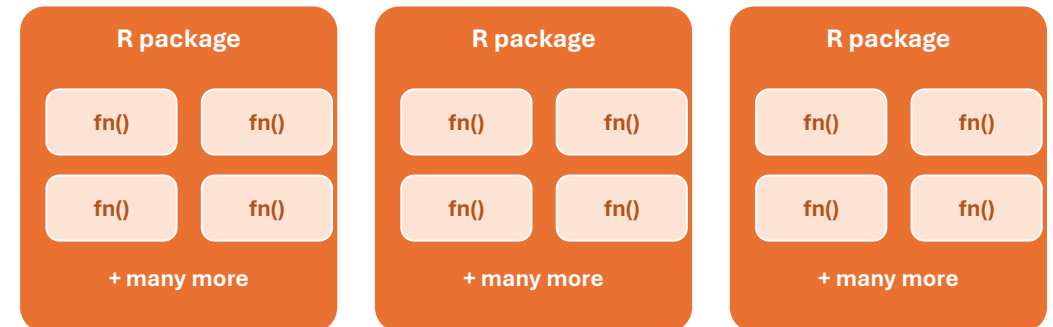
A Production Environment



Packages

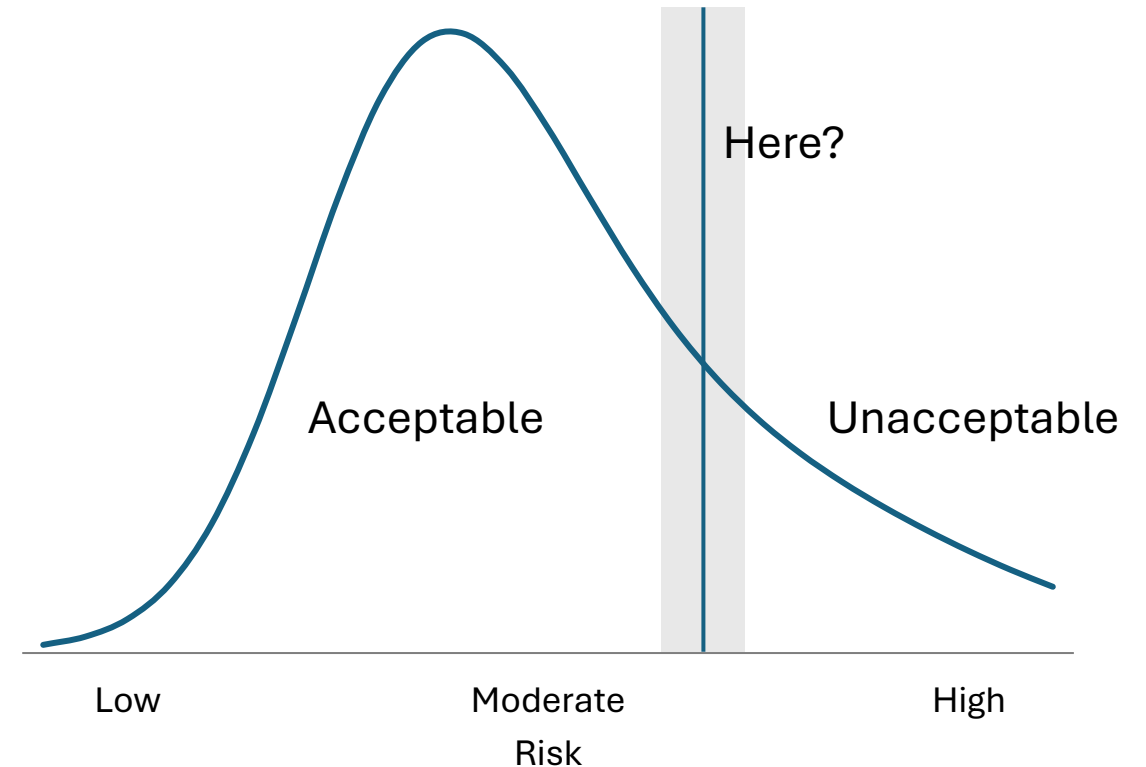
- They are open-source
- Developers should be devising and running unit tests
- Various metrics on package risk & “quality”.
- Do we (individual companies) REALLY need to re-test every function?
- If you don’t like it, **contribute** more or better test cases!

- Is risk package- specific or function-specific?
- Various risk assessment packages exist
- Is risk context-specific?



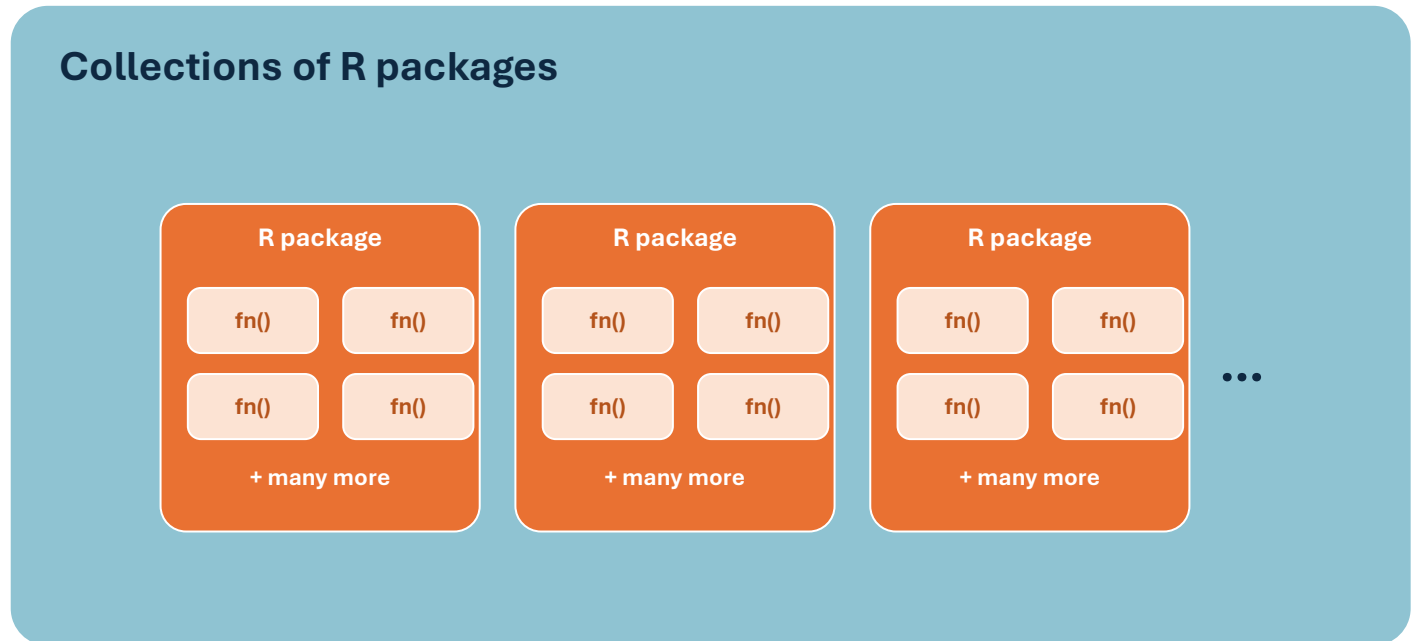
Collections of R packages

- What is an acceptable distribution of risk for our package set?
- How do we define “acceptable” and “unacceptable”?
- Are there some risk criteria that are more important than others?
- Any “show-stoppers”?



Collections of R packages

- Do risk assessment
- Consider risk distribution.
- Compromises may be necessary:
 - Essential but higher risk
 - What if dependencies are high risk?

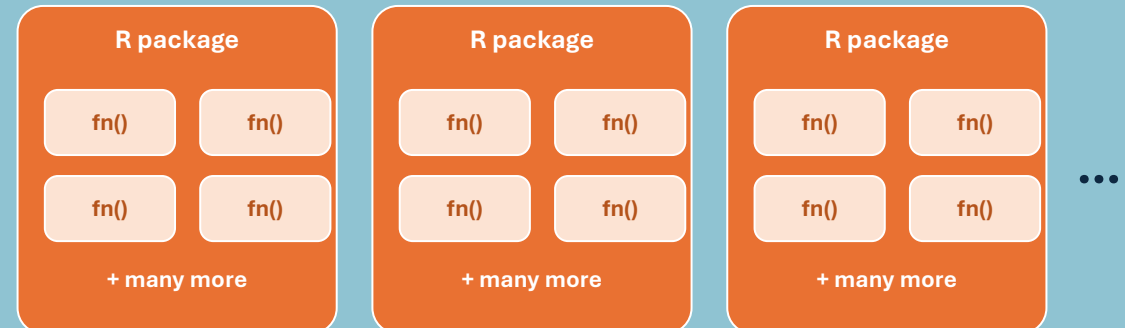


Collections of R packages

- CRAN snapshot date implies packages work well together
- How do you “bake in” your CRAN snapshot date?
- What do you do when colleagues need something “extra”?

- Use CRAN snapshots
- Posit Public Package Manager
- Use GitHub SHA commits
- Use {renv} or rv.
- Use {logrx}

Collections of R packages

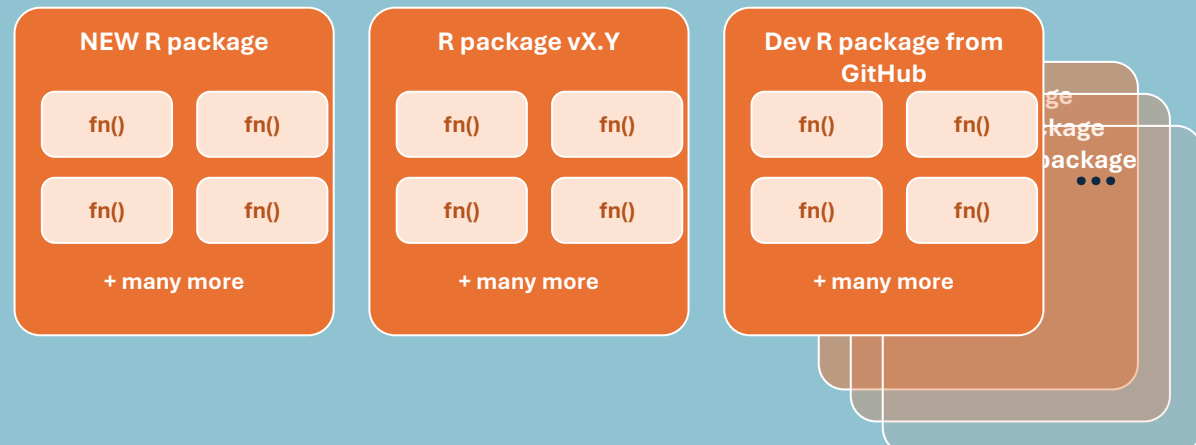


DANGER!

- Your colleagues come to Posit::Conf. They attend a workshop
- They install <WHO KNOWS?> from <NO IDEA> to do the workshop.
- They then return to work...

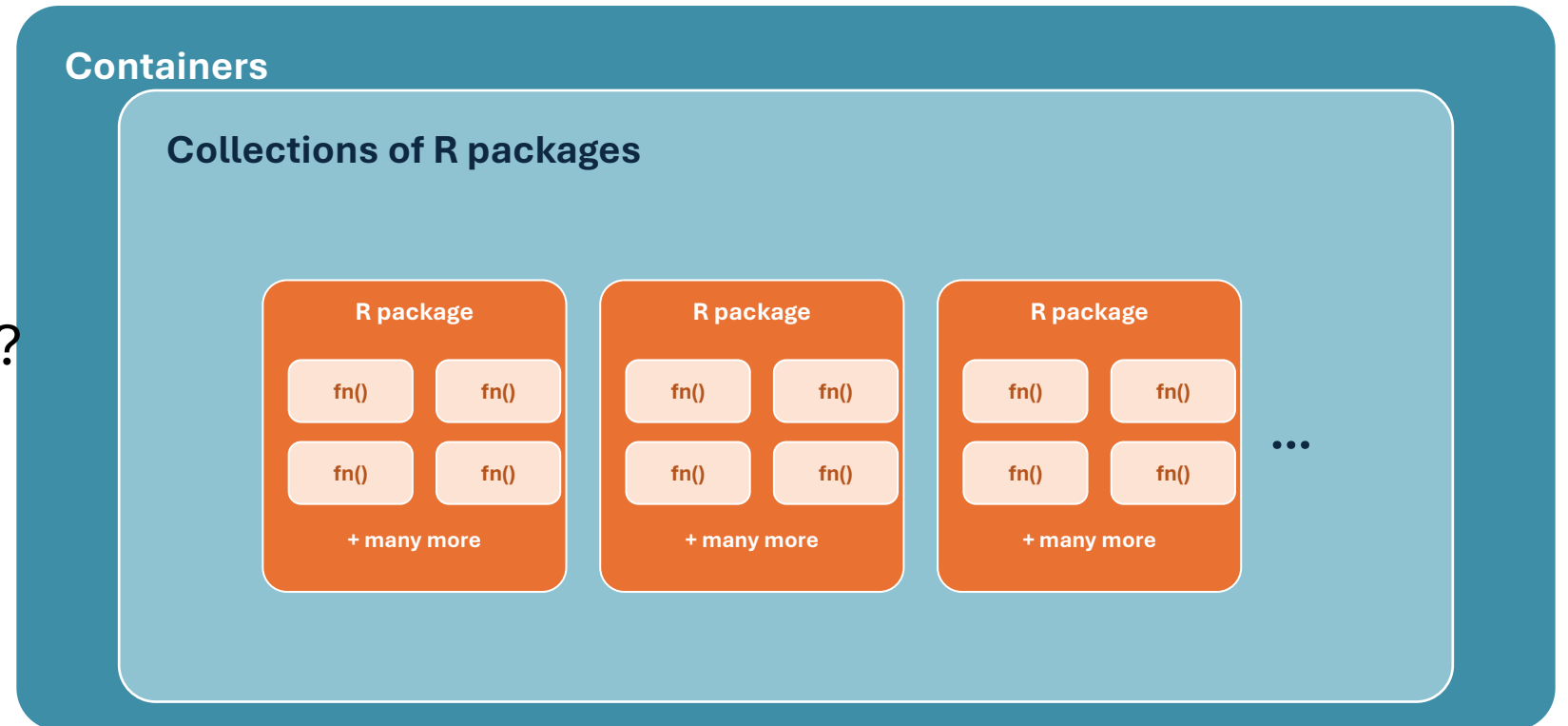


Collections of R packages



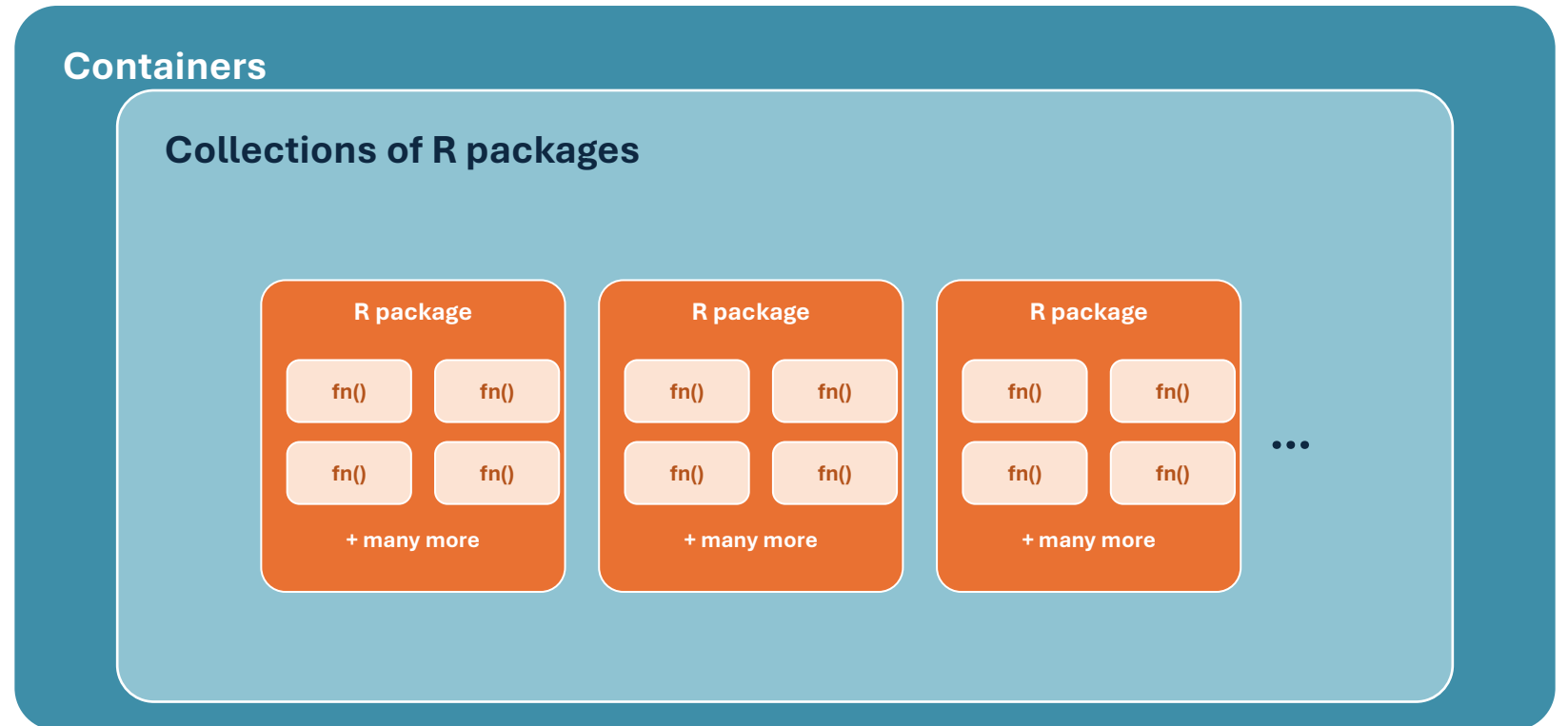
Let's start to bring it all together...

- Containers prevent additional work for re-validation
- Build it once, deploy it wherever you need it.
- Single source
- Controlled
- Immutable?
- Dependencies?
 - Quarto
 - Stan
 - Compilers



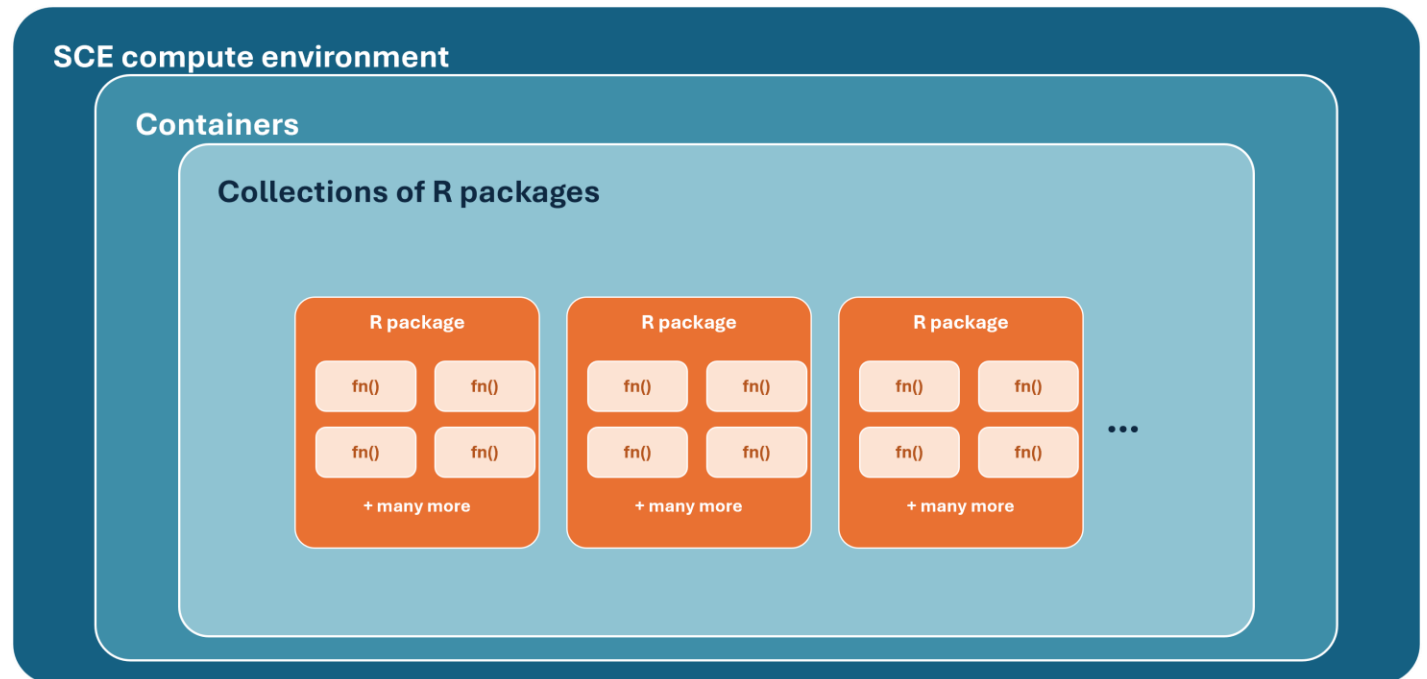
Containers = a new unit

- Containers are a reproducible unit.
- In 5 years... Spin up the old container.
- Need to verify

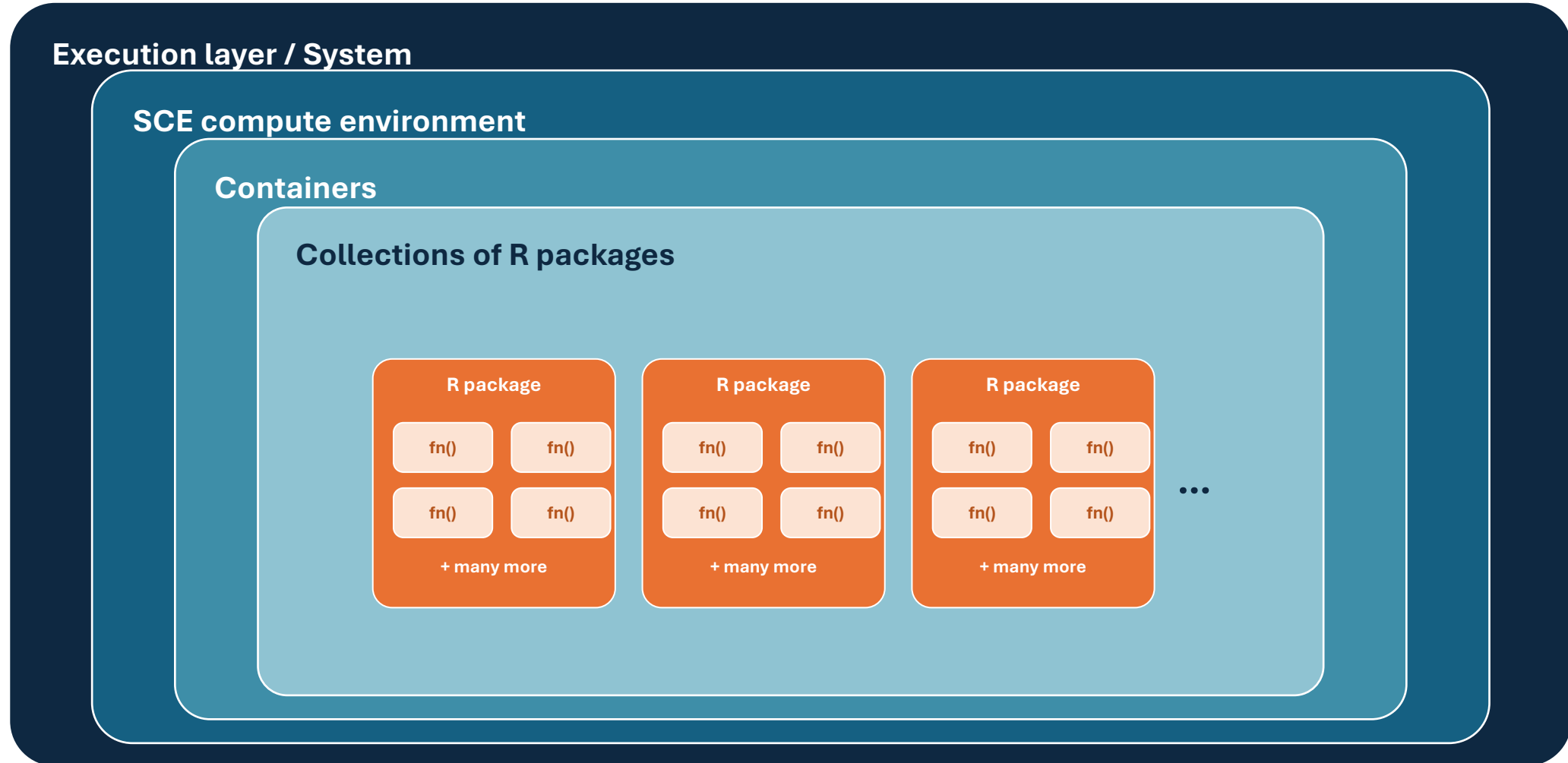


The SCE deployed container

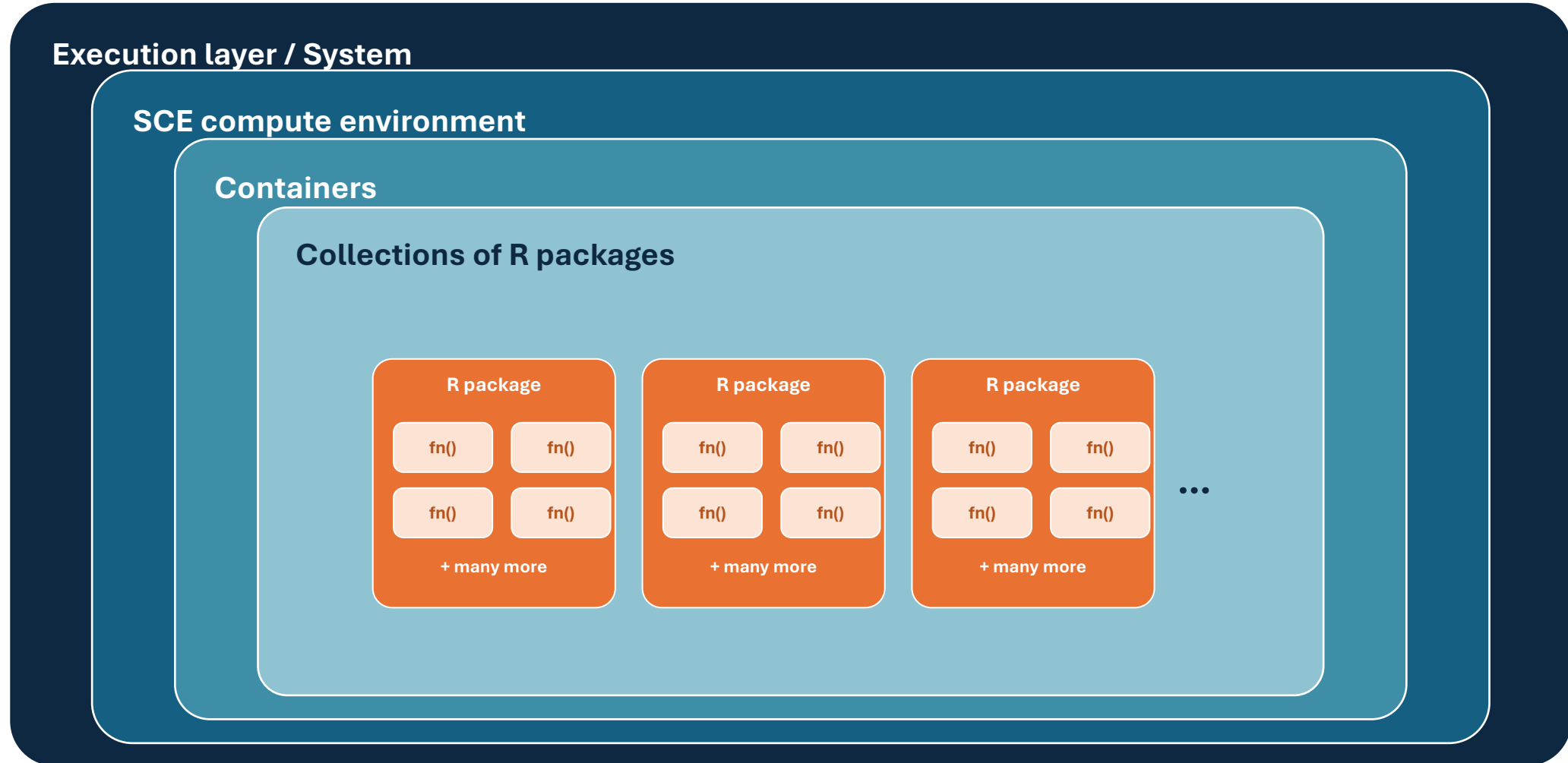
- Do you test and document at this level?
- It's not quite the full Production environment, but this defines the environment in which you're running your tools (from your platform / system).
- Tiers of deployment
 - Dev
 - Test
 - Stage
 - Prod



Now let's put it all together...



What levels do ***YOU*** test?



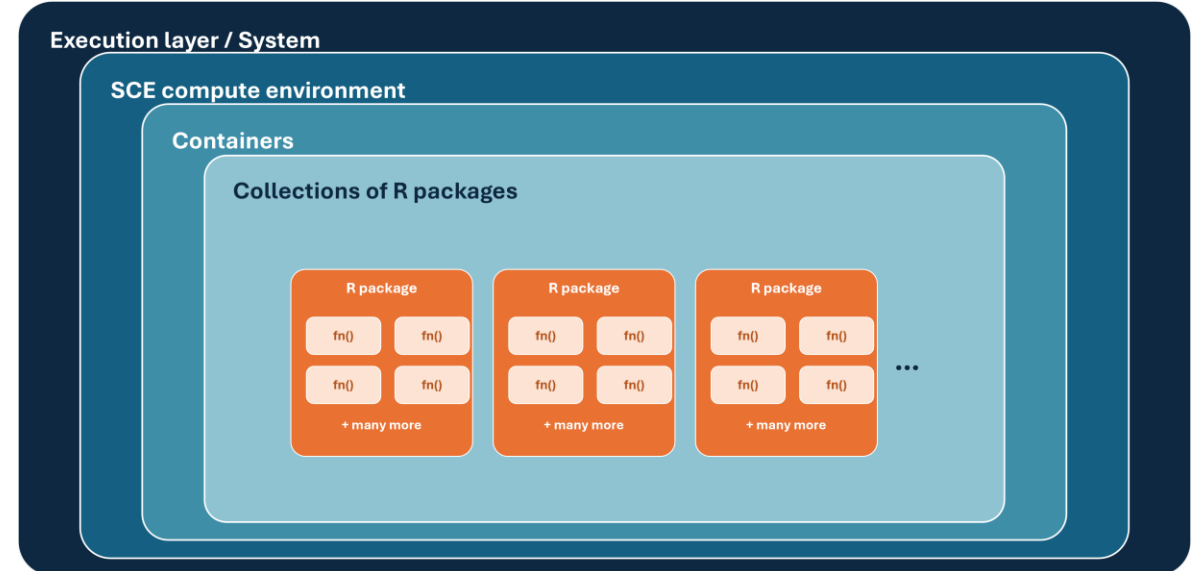
Every change should invoke testing

If your environment is held under change control...

...then you should be testing with every change.

...and documenting the tests and test findings.

- How do we test and document this?
- From our system / platform?
- Can we automate it?
- CI/CD would be nice...



Questions for your group to consider:

- What do you do for package validation? And how?
 - Is it risk-based? Is it context-specific?
 - Do you contribute findings back to package developers via Issues?
- Do you assess the (distribution of) risk of a collection of R packages?
- How do we ensure end-to-end validation, reproducibility, longevity?
- Can you ensure that older containers still work?
- How often do we re-qualify / re-validate / re-document the entire system?
- What is the smallest change that would trigger re-validation?
- What can we isolate to make re-validation easier? And how?